University of Massachusetts, Amherst
CICS
COMPSCI 683: Artificial Intelligence

Assignment 2: Informed Search

Important Instructions:

- Please upload your solution to Gradescope.
- You may work with a partner on your homework; however, if you choose to do so, **you must** write down the name of your partner on the assignment. In addition, you **may not submit similar/identical solutions**.
- Provide a mathematical proof/computation steps to all questions, unless explicitly told not to. Solutions with no proof/explanations (even correct ones!) will be awarded no points.
- Follow the guidelines on writing mathematical proofs provided on Canvas. Remember: we grade your proofs on correctness and clarity, not length. We will deduct points for incorrect claims, even if the proof as a whole is correct. For (a dramatic) example, if you conclude your proof with $1 + 1 = 4$, expect a grade deduction.
- You may look up combinatorial inequalities and mathematical formulas online, but please do not look up the solutions. Most problems can be found in research papers/online; if you happen upon the solution before figuring it out yourself, mention this in the submission. **Unreferenced copies of online material will be considered plagiarism.**

1. (15 points) Suppose that we are given an admissible heuristic function h. Consider the following function

$$h'(n) = \begin{cases} h(n) & \text{if } n \text{ is the initial state } s \\ \max\{h(n), h'(n') - c(n', n)\} & \text{where } n' \text{ is the predecessor node of } n \end{cases}$$

   where $c(n', n) \triangleq \min_a c(n', a, n)$. Prove that h' is consistent.

2. (25 points) Consider a search problem over a graph $G = (N, E, C)$, where N are nodes, E are directed edges between nodes, and the weight of an edge $e \in E$ is given by $c(e)$, where $c(e) > 1$, for all $e \in E$. We assume that there is a unique goal state g.
   (a) (5 points) Consider the heuristic h_1 that counts the number of edges between a given state s and the goal state g. Show that h_1 is consistent.
   (b) (4 points) Find a consistent heuristic h_2 that dominates the heurisic h_1. The heuristic h_2 cannot be equal to h_1, nor can it be the optimal heuristic h^* that computes the true cost of reaching g from the state s.
   (c) (8 points) Suppose that we add edges between states in the graph G, but keep the heuristic h_1 unchanged. Is h_1 still admissible and consistent? Prove this claim or give a counterexample.
   (d) (8 points) Suppose that we remove edges from the graph G, but keep the heuristic h_1 unchanged. Is the heuristic still admissible and consistent? Prove this claim or give a counterexample.

3. (20 points) Answer the following questions.
   (a) (10 points) Prove that if h is consistent, then it must be admissible (remember that for any goal state t, $h(t) = 0$).

(b) (10 points) Suppose that h, h' are heuristics such that h dominates h'. Show that if h is admissible, then so is h'; does the same claim hold for consistent heuristics? I.e. if h is consistent and dominates a heuristic h', is h' consistent as well? Prove or provide a counterexample.

4. (20 points) You have learned before that A* using graph search is optimal if $h(n)$ is consistent. Does this optimality still hold if $h(n)$ is admissible but inconsistent? Using the graph in Figure 1, let us now show that A* using graph search returns a sub-optimal solution path (S, B, G) from start node S to goal node G with an admissible but inconsistent $h(n)$. We assume that $h(G) = 0$.

   Give nonnegative integer values for $h(A)$ and $h(B)$ such that A* using graph search returns the non-optimal solution path (S, B, G) from S to G with an admissible but inconsistent $h(n)$, and tie-breaking is not needed in A*.

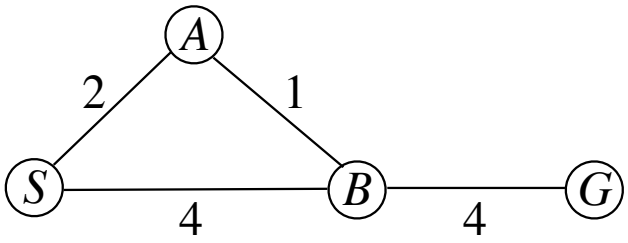

Figure 1: Graph for heuristic search.

5. (20 points) Consider the minimax search tree below (Figure 2). In the figure, black nodes are controlled by the MAX player, and white nodes are controlled by the MIN player. Payments (terminal nodes) are squares; the number within denotes the amount that the MIN player pays to the MAX player (an amount of 0 means that MIN pays nothing to MAX). Naturally, MAX wants to maximize the amount they receive, and MIN wants to minimize the amount they pay.

   Suppose that we use the α-β pruning algorithm. Consider the minimax tree given in Figure 2.

   (a) (10 points) **Assume that we iterate over nodes from right to left**; what are the arcs that are pruned by α-β pruning, if any?

   (b) (10 points) Does you answer change if we iterate over nodes from **left to right**?

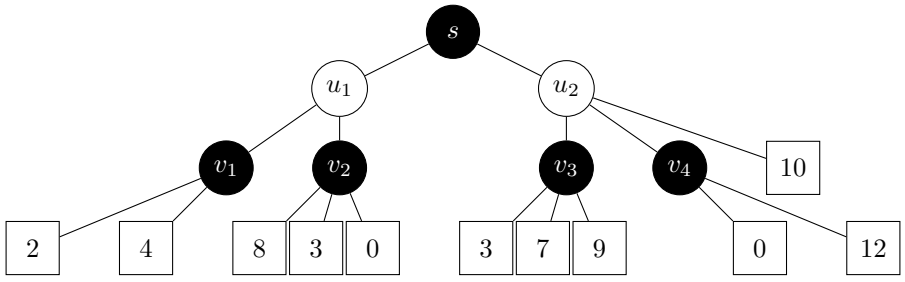

Figure 2: A minimax search tree.

6. (Optional, bonus points) In this question, we will formally prove that the minimax algorithm described in class produces an equilibrium. An extensive form game is defined by a set of nodes V and a set of directed edges E, defining a tree. The root of the tree is the node r. Let $V_{\max}$ be the set of nodes controlled by the MAX player

and $V_{\min}$ be the set of nodes controlled by the MIN player. We often refer to the MAX player as player 1, and to the MIN player as player 2. Furthermore, let $desc(v)$ be the descendants of a node v in the search tree. A *strategy* for the MAX player is a mapping $s_1 : V_{\max} \to V$; similarly, a strategy for the MIN player is a mapping $s_2 : V_{\min} \to V$. In both cases, $s_i(v) \in chld(v)$ is the choice of child node that will be taken at node v. We let $\mathcal{S}_1, \mathcal{S}_2$ be the set of strategies for the MAX and MIN player, respectively.

The leaves of the minimax tree are *payoff nodes*. There is a payoff $a(v)$ associated with each payoff node v, where $a(v)$ is the amount that the MAX player receives and $-a(v)$ is the amount that the MIN player receives if the node v is reached. The MAX player wants to maximize their payoff, and the MIN player wants to minimize the amount that they pay. More formally, the utility of the MAX player from v is $u_{\max}(v) = a(v)$ and the utility of the MIN player is $u_{\min}(v) = -a(v)$. The utility of a player from a pair of strategies $s_1 \in \mathcal{S}_1, s_2 \in \mathcal{S}_2$ is simply the utility they receive by the leaf node reached when the strategy pair (s_1, s_2) is played.

Definition 1 (Nash Equilibrium)**.** A pair of strategies $s_1^* \in \mathcal{S}_1, s_2^* \in \mathcal{S}_2$ for the MAX player and the MIN player, respectively, is a *Nash equilibrium* if no player can get a strictly higher utility by switching their strategy. In other words:

$$\forall s \in \mathcal{S}_1 : u_1(s_1^*, s_2^*) \geq u_1(s, s_2^*);$$
$$\forall s' \in \mathcal{S}_2 : u_2(s_1^*, s_2^*) \geq u_2(s_1^*, s')$$

Definition 2 (Subgame)**.** Given an extensive form game $\langle V, E, r, V_{\max}, V_{\min}, \vec{a} \rangle$, a subgame is a subtree of the original game, defined by some arbitrary node v set to be the root node, and all of its descendants (i.e. its children, its children's children etc.), denoted by $desc(v)$. Leaf nodes and node control are the same as in the original extensive form game.

Definition 3 (Subgame-Perfect Nash Equilibrium (SPNE))**.** A pair of strategies $s_1^* \in \mathcal{S}_1, s_2^* \in \mathcal{S}_2$ is a sub-perfect Nash equilibrium if it is a Nash equilibrium for any subtree of the original game tree.

(a) Prove that the minimax algorithm shown in class outputs a subgame-perfect Nash equilibrium.

(b) Consider a node v in an adversarial search problem. Assume that v is controlled by the MAX player (Player 1). Prove that α-β pruning only removes subtrees of v that offer Player 1 a payoff that is weakly lower than their payoff in the subtree rooted in v under a subgame-perfect Nash equilibrium.